Preprint | Software v0.5.1 | November 2026

# STEMKit: A client-side toolkit for computational chemistry and scientific data analysis

Olanrewaju M. Daramola
Independent Researcher | lanrelangmuir@gmail.com

**Abstract**

STEMKit is a suite of 18 browser-based tools for computational chemistry and scientific data analysis, built on `stemkit-core`, a JavaScript library of parsing and numerical routines with no DOM dependency. All computation runs inside the user's own browser: the data analysed never leaves it, no account is required and nothing is installed, so the tools stay usable for unpublished or confidential data. The same modules run under Node.js, so an interactive analysis can be captured as a version-pinned script. The 42 domain modules carry 4989 tests validated against SciPy, NumPy, statsmodels, pandas, matplotlib, GROMACS, PLUMED, LAMMPS and physical invariants rather than against the implementation itself. That validation exposed four defects that had been altering reported results, among them $p$-values floored at zero and the mass of haem iron taken as fluorine's. All four are corrected and covered by regression tests.

**Keywords:** Computational chemistry; Molecular dynamics; Client-side computation; GROMACS; LAMMPS; PLUMED; Reproducibility; Data privacy

**Table 1.** Software availability.

| | |
|---|---|
| **Version** | `v0.5.1` |
| **Source** | `https://github.com/LD-Shell/stemkit` |
| **Archive** | Zenodo, concept DOI `10.5281/zenodo.21543112`, which resolves to the current release |
| **Licence** | MIT |
| **Languages** | JavaScript (ECMAScript 2020 modules), HTML5, CSS3; Node.js; Jest; Tailwind CSS |
| **Requirements** | Browser tools: any current browser with ECMAScript modules and the `FileReader` API; nothing to install. Library: Node.js $\geq$ 18, on npm as `stemkit-core`, with no runtime dependencies (jStat, Papa Parse, regression.js and bibtex-parse-js are bundled) |
| **Documentation** | `https://github.com/LD-Shell/stemkit#readme`; API in `src/core/README.md`; tools at `https://stemkit.net` |
| **Tutorials** | Narrated video tutorials for every research tool, shown on each tool's page and collected at `https://www.youtube.com/playlist?list=PLAqgfbqFf32M` |
| **Support** | `https://github.com/LD-Shell/stemkit/issues` |

## 1 Motivation and significance

The software landscape of computational chemistry is well served at its extremes. Simulation engines such as GROMACS [1] and LAMMPS [2] are mature, extensively validated, and

actively maintained; analysis frameworks such as MDAnalysis [3] provide comprehensive programmatic access to trajectory data. Between these poles lies a substantial volume of routine work that neither category serves comfortably.

Consider the interval between completing a simulation and reporting it. A researcher must inspect the RMSD to confirm equilibration, extract a radius of gyration, convert a structure between the nanometre convention of GROMACS and the Å convention of the Protein Data Bank, compare replicate measurements statistically, digitise a figure from a paper for comparison, abbreviate journal titles in a reference list, and compose a scheduler script requesting an appropriate resource allocation. None of these operations is intellectually demanding. All of them are error-prone when performed by hand, and all of them are performed repeatedly.

### 1.1 The friction problem

Three classes of solution exist, and each imposes a characteristic cost.

*Local scripting.* A Python environment with MDAnalysis, NumPy and SciPy addresses every task listed above and is the correct instrument for sustained analytical work. Its cost is environmental. Managed institutional workstations frequently prohibit package installation; conda environments drift; and a script written six months ago may no longer execute against the current interpreter. For a researcher whose primary expertise lies in chemistry rather than software engineering, this cost is not negligible.

*Web services.* Server-side tools eliminate installation entirely but require data transmission. For unpublished results, material subject to a confidentiality agreement, or work under embargo, this is frequently prohibited outright. Even where permitted, the transfer creates a record whose retention policy the researcher does not control.

*Desktop applications.* Graphical applications avoid both problems but introduce a third: they are difficult to script. An analysis performed by clicking cannot readily be re-executed, version-controlled, or subjected to continuous integration. In practice such an analysis is difficult to reproduce, even when every step was recorded at the time.

### 1.2 The client-side position

The browser is a capable computational environment. `FileReader` grants access to local files without transmission, and the engines are fast enough for the volumes involved: the parser sustains roughly $7 \times 10^5$ rows per second, reading a two-million-row, 53 MB `.xvg` in 2.9 s, and parses a million-atom PDB and returns its centre of mass in 1.3 s. The ceiling is memory rather than arithmetic, since `FileReader` materialises the file as a string; a few hundred megabytes is practical, beyond which streaming would be required. ECMAScript modules make it possible to organise that code properly, instead of accumulating scripts.

STEMKit is built around these constraints. No server participates in the computation, so there is nothing to which data could be transmitted and users are not asked to take a privacy policy on trust. Two features make a network request, and only when asked to: the DOI lookup sends the DOIs entered to doi.org to retrieve their bibliographic records, and the structure inspector can fetch an entry from the Protein Data Bank by its four-character identifier. Neither sends a file or a result. Every asset is vendored in the repository: there is no content-delivery network, no analytics and no telemetry, so not even a request log is generated elsewhere, and a local clone runs with the network disconnected. The reproducibility property derives from a separate architectural decision: the numerical core is extracted into modules with no dependency on the browser, so the identical code that executes behind the interface also executes under Node.js.

### 1.3 Experimental setting

The software is used in two ways sharing one implementation. Interactively, the user opens a tool page at `https://stemkit.net` (or a local clone), selects a local file and reads the result in the page; the file is read through `FileReader` and never uploaded. Programmatically, the same routines are imported from `stemkit-core` in a Node.js script, the path taken once an analysis is to be fixed, version-pinned and re-run. The seven plotting tools (the XVG visualiser, plot builder, curve fitter, plot digitiser, error-bar generator, statistics calculator and outlier detector) and the MD workflow generator's PLUMED run analysis additionally emit a standalone matplotlib script that regenerates the figure from the same data, and the data cleaner a pandas script that repeats its cleaning steps, so an interactive result can be reproduced offline without the browser.

Related work is cited where the corresponding functionality is described in Section 2: GROMACS [1], LAMMPS [2] and PLUMED [4] for the file formats, MDAnalysis [3] as the reference framework, 3Dmol.js [5], whose atom-selection syntax the `selection` module mirrors, SciPy [6], NumPy [7] and statsmodels [8] for validation, the primary statistical literature [9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22] for the implemented tests, CODATA 2018 [23] for conversion factors, and standard practice for the incomplete beta and gamma functions [24].

## 2 Software description

The 18 research tools are the subject of this paper. Three further browser utilities (a Pomodoro timer, a decision matrix and a kinetics sandbox) are distributed from the same repository as general workflow aids; they are not part of the scholarly contribution and are not described here, though they share the presentation layer and so appear in the page counts below.

### 2.1 Software architecture

The original implementation followed the pattern common to browser tooling: each of the 21 tool pages consisted of an HTML document paired with a single JavaScript file combining event handling, rendering and computation. This arrangement is expedient during development and progressively obstructive thereafter. Computational logic entangled with DOM manipulation cannot be tested without a browser, cannot be reused across tools, and cannot be invoked from a script.

The refactoring applied the *strangler fig* pattern: for each tool the mathematical and parsing logic was extracted into a pure module beneath `src/core/`, a test suite written against it, and the original file reduced to DOM wiring importing the new functions. The interface is unchanged; the computation beneath it is now independently addressable. Figure 1 shows the resulting layering and Table 2 enumerates the modules.

Four third-party libraries are vendored in the repository for the core: jStat, Papa Parse, regression.js and bibtex-parse-js. The pages also vendor Plotly, KaTeX, 3Dmol.js and MathLive, which the core does not use. Each of the four ships as a Universal Module Definition (UMD) bundle, which detects its host at run time and either assigns `module.exports` under CommonJS or attaches a global to `window`. Neither path is reachable from a plain ECMAScript module: a UMD file has no `export` statements, so a direct import yields no bindings, while `import { createRequire } from 'module'` is a hard *resolution* failure in a browser, occurring before any code runs and so uncatchable. The dynamic `await import()` does reject catchably, but injection was preferred because the bundles must be available synchronously. Rather than branch on the execution environment inside every module, which would render the core untestable in

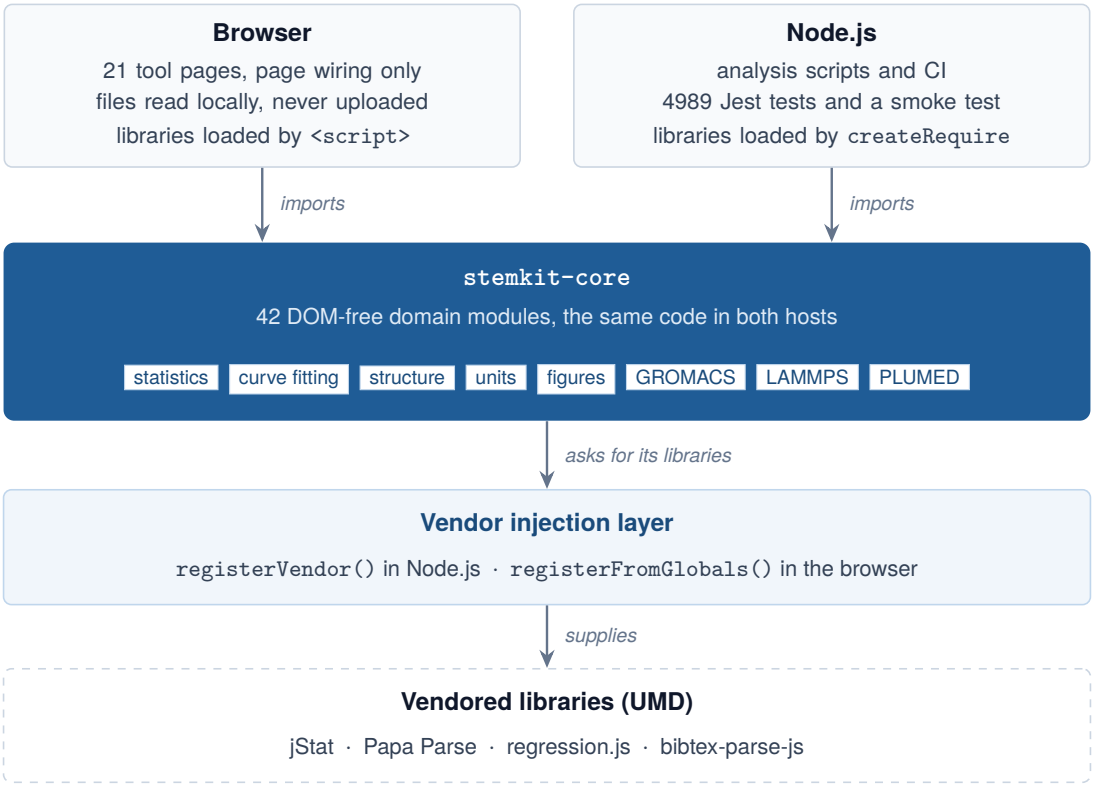



**Figure 1.** Architecture of STEMKit. Both hosts execute the identical core modules; the only environment-specific code is the registration call that supplies the vendored libraries.

one of its two targets, the library declares the libraries it requires and each host registers them at initialisation: under Node.js the package's entry point, in a browser the page (Section 2.3). Modules request their dependencies lazily, at the point of use rather than at import time, so importing any module never fails merely because an unrelated library has not yet been registered.

The test suite comprises 4989 tests in 66 suites across the 42 domain modules and the version module, with 89.5% statement and 93.6% line coverage of `src/core/`. Its governing principle is that numerical results are validated against *independent* references rather than against the implementation under test, since a test written from the same source as the code confirms only internal consistency. Reference values were obtained from SciPy [6] for t-tests, ANOVA, post-hoc comparisons, correlation, non-parametric tests, quantiles and the studentised range distribution, and from its `curve_fit` for non-linear fits, their standard errors and confidence bands; from statsmodels [8] for Welch's ANOVA and multiple-comparison correction; from NumPy [7] for regression coefficients and descriptive statistics; from `scipy.constants` for every unit conversion factor; from pandas [25] for the data cleaner's scripts, whose output is compared with the page's cell by cell; from matplotlib [26] for box-plot whiskers and for the axis limits the generated figure scripts produce; from GROMACS 2025 [1], through `grompp`, `make_ndx` and `select`, for `.mdp` checks, index groups and selections; from PLUMED 2.9 to 2.11 [4] for the generated inputs, and from its `sum_hills` for free-energy surfaces; from LAMMPS 29 Aug 2024 [2] for data files, input checks and the generated inputs; and from physical invariants for structural geometry, specifically the molecular weight and centre of mass of water, the orthonormality of rotation matrices, the preservation of interatomic distances under rotation, and round-trip fidelity through every supported file format. An end-to-end smoke test additionally exercises one representative path through 28 of the 44 modules the package

**Table 2.** Modules of `stemkit-core`. Vendored dependencies are supplied by injection; modules marked *none* operate without any third-party code. Related modules share a row, and a row's tests include those of the page scripts built on it (439 in all).

| Module | Domain | Tests | Dependency |
|---|---|---|---|
| `gromacs-mdp`, `gromacs-ndx` | GROMACS `.mdp` settings and checks, index groups | 1168 | none |
| `plumed`, `plumed-syntax`, `plumed-parse`, `plumed-atoms`, `plumed-run` | PLUMED input generation and checking, atom groups, run files | 644 | none |
| `expression`, `nonlinear-fit`, `fit-python`, `plot-style` | Typed equations, Levenberg–Marquardt fitting, fit scripts | 459 | none |
| `lammps-reference`, `lammps-input`, `lammps-data`, `lammps-workflow` | LAMMPS commands, input checks, data files, run inputs | 355 | none |
| `bibtex` | Reference deduplication, sanitising | 236 | bibtex-parse-js |
| `statistics`, `statistics-figure` | Inferential and descriptive statistics | 193 | jStat |
| `data-cleaning`, `data-cleaning-python` | Tabular transformation, pandas script | 183 | Papa Parse |
| `figure`, `figure-python` | Figure description, matplotlib script | 167 | none |
| `structure` | Molecular geometry, PDB/GRO/XYZ, rigid transforms and their scripts | 165 | none |
| `xvg-parser` | GROMACS/PLUMED trajectory data | 162 | none |
| `slurm` | SLURM job scripts, resource checks | 123 | none |
| `plumed-analysis`, `plumed-analysis-figures` | COLVAR and HILLS analysis, free-energy surfaces | 121 | none |
| `iso4` | ISO 4 word-level abbreviation (LTWA) | 119 | none |
| `error-bars`, `error-bars-figure` | Group summaries, error bars | 107 | jStat |
| `units` | Physical unit conversion | 106 | none |
| `selection` | Atom selection, spatial queries | 94 | none |
| `curve-fitting` | Least-squares regression | 81 | regression.js |
| `outliers`, `outliers-figure` | Anomaly detection | 78 | jStat |
| `scheduler` | Scheduler directives and launchers | 73 | none |
| `plot-builder` | Plot builder tables and figures | 62 | none |
| `latex` | Table generation and escaping | 61 | none |
| `digitizer` | Figure digitisation | 57 | none |
| `pdf` | PDF export of figures | 56 | none |
| `journals` | Whole-title journal abbreviation | 53 | none |
| `version` | Release version and date | 5 | none |
| `zip` | Archives of generated files | 4 | none |
| (no module) | Page scripts and site tools with no module under them: the Decision Matrix, the method and how-to tabs, the sitemap and page checks | 57 | none |
| **Total** | | **4989** | |

exposes, the injection layer among them, against a real installation, detecting the class of failure that unit tests cannot: a broken aggregate export, a misconfigured module type declaration, or a vendored bundle that fails to load.

## 2.2 Software functionalities

Table 3 maps each tool to the modules it calls; the subsections below describe the numerics grouped by domain.

Only the structure inspector still computes in its page script. The `selection` module is a tested extraction of its selection language; the page does not call it yet.

### 2.2.1 Trajectory and collective-variable data

GROMACS writes analysis output in the Grace format, in which `@` directives carry the plot title, axis labels and per-series legends alongside the numeric records. A parser that skips all non-numeric lines discards precisely the metadata identifying which column holds which

**Table 3.** The 18 research tools and the core modules behind them. Three further pages (Pomodoro timer, decision matrix, kinetics sandbox) are workflow aids and are excluded. The plotting tools also share one plot area, built on `figure`, `figure-python`, `plot-style` and `pdf`.

| Tool | Module(s) | Purpose |
|---|---|---|
| XVG visualiser | `xvg-parser` | Plot GROMACS/PLUMED series with recovered axis metadata |
| Structure inspector | none (page script) | Geometry, mass breakdown, atom selection and search by residue or species, simulation box, 3D view |
| Coordinate manipulator | `structure` | Translate, rotate by angles or by hand, re-box; PDB/GRO/XYZ interconversion; the steps as equations and a Python script |
| MD workflow generator | `slurm`, `scheduler`, `gromacs-mdp`, `gromacs-ndx`, `lammps-reference`, `lammps-input`, `lammps-data`, `lammps-workflow`, `plumed`, `plumed-syntax`, `plumed-parse`, `plumed-atoms`, `plumed-analysis`, `plumed-analysis-figures`, `zip` | GROMACS and LAMMPS inputs written and checked, index groups, batch scripts on four schedulers; PLUMED inputs, checks and run analysis |
| Statistics calculator | `statistics`, `statistics-figure` | t-tests, one-way and Welch ANOVA with post-hoc comparisons, correlation, non-parametric tests, assumption checks, test recommendation |
| Error-bar generator | `error-bars`, `error-bars-figure` | Group summaries, error bars, significance annotation |
| Outlier detector | `outliers`, `outliers-figure` | Tukey fences, modified Z-score, Grubbs' test |
| Curve fitter | `expression`, `nonlinear-fit`, `fit-python`, `plot-style` | Typed equations fitted by Levenberg–Marquardt, with standard errors and confidence bands |
| Plot digitiser | `digitizer` | Pixel-to-data recovery with log-axis and uncertainty handling |
| Plot builder | `plot-builder`, `figure` | Publication figures (lines, bars, histograms, box and violin plots, heatmaps, contours) on one or several panels, at a set print size and resolution, with matplotlib script export |
| Data cleaner | `data-cleaning`, `data-cleaning-python` | Missing values, duplicates, filtering and column transforms, with a pandas script |
| Scientific converter | `units` | 64 units in ten categories plus temperature, CODATA-sourced |
| BibTeX sanitiser | `bibtex` | Field removal, page ranges, braces around capitals in titles |
| BibTeX deduplicator | `bibtex` | Union-find over normalised DOIs and titles; entries with different DOIs are never merged |
| DOI to BibTeX | `bibtex` | Lookup at doi.org with field filtering |
| Journal abbreviator | `journals`, `iso4` | Whole-title dictionary, then ISO 4 word-level fallback |
| Visual LaTeX tables | `latex` | LaTeX and Markdown tables with correct escaping |
| Equation formatter | `latex` | Formula entry and LaTeX output |

observable; `xvg-parser` recovers it with the matrix. Malformed records are counted and reported rather than silently dropped, so a truncated trajectory is visible instead of quietly shortening the analysis. Two further details of the format are handled explicitly: the Grace multi-set separator `&` is treated as a delimiter rather than parsed as data, and records containing `NaN` or `Infinity` are rejected whole, since one non-finite entry would otherwise contaminate every downstream statistic.

PLUMED [4] writes `COLVAR` files whose `#!` headers follow a different convention but share Grace's comment character, so the same parser handles both without modification. An explicit test verifies this rather than leaving it to assumption.

### 2.2.2 Structure files and molecular geometry

The `structure` module parses the three coordinate formats in routine use for classical simulation: PDB and GROMACS `.gro`, both fixed-column, and whitespace-delimited XYZ. Fixed-column parsing is essential rather than pedantic: a residue name may legitimately be blank, and a whitespace-splitting parser would silently shift every subsequent field on that line. Unit handling is explicit throughout: PDB and XYZ express coordinates in Å, `.gro` uses nanometres, and every parse result carries its unit. Silently mixing the two is the most direct route to a structure that is wrong by a factor of ten.

Assigning an element to each atom is a prerequisite for any mass-weighted quantity, and PDB atom names are ambiguous by construction: `CA` denotes the $\alpha$-carbon in a protein residue and calcium in an ion record. The resolution implemented here proceeds in five stages.

1. An explicit element column, where present, takes priority.
2. A leading digit is a hydrogen-count prefix (`1HB`, `2HG1`) and is stripped before the symbol is read.
3. An unambiguous two-letter metal (`FE`, `ZN`, `SE`) is recognised wherever it appears, including within a host residue such as haem or selenomethionine.
4. Otherwise, a two-letter prefix whose second character is uppercase is a PDB remoteness indicator, and only the first letter is the element: `CA` in a protein residue is C$\alpha$.
5. As a final fallback, a two-letter symbol is trusted when the residue shares its name, the convention for monatomic ions.

Stages 2 and 3 correct defects present in the original implementation (Section 2.2.5). The resolution is validated against 25 atom names drawn from real structures, covering protein backbone and side-chain atoms, nucleic acid atoms, water, monatomic ions and metal centres. On this basis the module computes the geometric centroid, the mass-weighted centre of mass, the axis-aligned bounding box, and the radius of gyration

$$R_g = \sqrt{\frac{\sum_i m_i \, |\mathbf{r}_i - \mathbf{R}|^2}{\sum_i m_i}}, \qquad \mathbf{R} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}, \tag{1}$$

together with rigid-body rotation about an arbitrary pivot using intrinsic $Z$–$Y$–$X$ Euler angles. A rotation is also held as its matrix, as the single turn about one axis that it amounts to, and as the orientation of a three-dimensional view, so the coordinate manipulator lets a structure be turned by hand against a fixed cell and reports the angle between its principal axis and each coordinate axis before the turn is applied. Every step is recorded, and from that record the module writes each step's equation, the one transform $\mathbf{r}' = M\mathbf{r} + \mathbf{d}$ the steps compose to, and a Python script that repeats them; the script is run in the tests and must return the page's coordinates. Velocities present in a `.gro` file rotate with the frame but are never translated, so a round trip through rotation and export preserves the kinetic state.

### 2.2.3 Statistics, outliers and curve fitting

The `statistics` module implements the one-sample, paired, Welch's [9] and Student's t-tests, one-way ANOVA and Welch's ANOVA [10], Pearson and Spearman [11] correlation, and the Mann–Whitney $U$, Wilcoxon signed-rank and Kruskal–Wallis [12] tests. Welch's test is the default for independent samples, as it does not assume equal variances and loses negligible power when they are in fact equal. Every parametric test reports an effect size and, where

standard, a confidence interval. Assumptions are checked and surfaced rather than presumed: normality by the D'Agostino–Pearson $K^2$ omnibus test [17], homogeneity of variance by the median-centred Brown–Forsythe variant of Levene's test [18], chosen for its greater robustness to non-normality. `recommendTest` turns those checks into advice: from the design (independent, paired, one-sample or association) and the assumption checks it names the test to use and states why, and the caller decides whether to follow it. After a test on three or more groups, pairwise comparisons follow the same logic: Tukey's HSD in the Tukey–Kramer form for unequal group sizes [13, 14] after ANOVA, Games–Howell [15] after Welch's ANOVA, and Dunn's rank-sum test [16] with the Holm adjustment [19] after Kruskal–Wallis. Every analysis is accompanied by per-group descriptive statistics and a plot of the data. Table 4 maps each computed quantity to its numerical method and validation reference.

**Table 4.** Numerical methods and validation references.

| Quantity | Method | Validated against |
|---|---|---|
| Student / Welch $t$, $p$ | Regularised incomplete beta $I_x(a, b)$ | `scipy.stats.ttest_ind` |
| One-sample $t$ | Regularised incomplete beta $I_x(a, b)$ | `scipy.stats.ttest_1samp` |
| Welch–Satterthwaite $\nu$ | Closed form | SciPy, hand calculation |
| One-way ANOVA $F$, $p$ | Sum-of-squares decomposition; $I_x$ tail | `scipy.stats.f_oneway` |
| Welch's ANOVA $F$, $p$ | Precision-weighted means [10]; $I_x$ tail | statsmodels `anova_oneway` |
| Tukey HSD | Studentised range, Tukey–Kramer standard errors | `scipy.stats.tukey_hsd` |
| Games–Howell | Studentised range, per-pair Welch df | Formula, `scipy.stats.studentized_range` |
| Pearson $r$, CI | Fisher $z$ transform | `scipy.stats.pearsonr` |
| Spearman $\rho$, CI | Pearson on average ranks; Bonett–Wright interval [22] | `scipy.stats.spearmanr` |
| Mann–Whitney $U$ | Exact without ties when a group has at most 8 values; tie-corrected normal approximation otherwise | `scipy.stats.mannwhitneyu` |
| Kruskal–Wallis $H$ | Tie-corrected; $\chi^2$ tail | `scipy.stats.kruskal` |
| Dunn's test | Rank-sum $z$, Holm adjustment | Formula, statsmodels `multipletests` |
| Wilcoxon $W$ | Exact up to 50 pairs without ties or zeros; permutation up to 13 pairs; tie-corrected normal approximation otherwise | `scipy.stats.wilcoxon` |
| D'Agostino–Pearson $K^2$ | $\sqrt{b_1}$ [27] and $b_2$ [28] transforms | `scipy.stats.normaltest` |
| Levene $W$ | Brown–Forsythe (median-centred) | `scipy.stats.levene` |
| Quantiles | Linear interpolation (type 7) | `numpy.percentile` |
| Multiple comparisons | Holm–Bonferroni [19] | statsmodels `multipletests` |
| Grubbs $G$ | Inverted $t$ critical value | `scipy.stats.t.ppf` |
| Modified $Z$-score | Median/MAD, 0.6745 scaling [20] | `numpy` |
| Least-squares fitting | Normal equations (regression.js) | `numpy.polyfit` |
| Non-linear fitting | Levenberg–Marquardt on untransformed data | `scipy.optimize.curve_fit` |

Upper-tail probabilities are computed through the complementary form of the incomplete beta and gamma functions rather than as $1 - F(x)$. The two expressions are algebraically identical; numerically they are not. Once $F(x)$ rounds to unity in double precision, the subtraction cancels completely and the result is exactly zero. The consequence is not academic. For the one-way ANOVA used in the test suite ($F = 377.545$ on 2 and 21 degrees of freedom), the naive

form returns $p = 0$, whereas

$$p = I_{\frac{d_2}{d_2 + d_1 F}}\left(\tfrac{d_2}{2}, \tfrac{d_1}{2}\right) = 3.461\,174\,738\,289\,425\,5 \times 10^{-17}, \tag{2}$$

reproducing `scipy.stats.f.sf` to full double precision. A continuous test statistic cannot produce $p = 0$, so that value in published output indicates a numerical fault rather than an unusually strong effect. The same treatment is applied to the Student $t$, $\chi^2$ and normal tails; for $\chi^2$ the upper incomplete gamma function is evaluated directly, by its continued fraction [24], wherever the subtraction would cancel. Beyond $|z| = 8$ the vendored `erfc` underflows and a documented asymptotic expansion takes over, accurate to $3.5 \times 10^{-6}$ relative at the switchover and $1.6 \times 10^{-10}$ by $z = 30$; at $z = 38$ it returns $5.8 \times 10^{-316}$, where `2*scipy.stats.norm.sf(38)` has already underflowed. Its own floor lies near $z = 38.5$, and that bound rather than any general principle is what limits how small a reported $p$ can be.

One limitation of `curve-fitting` is documented rather than fixed, and it applies to two of the three transformed models rather than to all of them. The vendored regression.js library fits the *power* model by regressing $\ln y$ on $\ln x$ unweighted, so the residuals minimised are those in logarithmic space and small $y$ values carry more influence than a direct fit would give them. The *exponential* model also works in log space but weights each point by $y$, which is the standard correction for log-transform bias and keeps the result closer to a direct non-linear fit. The *logarithmic* model carries none of this bias: it regresses $y$ on $\ln x$, so only the predictor is transformed and the residuals minimised are those in $y$, exactly as for any model linear in its parameters. Where the bias does apply it is material. On an exactly doubling series the distinction vanishes, since the log-space residuals are identically zero and every least-squares variant recovers $\ln 2$; it appears only once the data are perturbed. For the series $(1, 2.0)$, $(2, 4.1)$, $(3, 8.2)$, $(4, 16.1)$, $(5, 32.3)$ the weighted fit returns a growth rate of 0.690 216 against 0.693 167 for unweighted log-space least squares, with $\ln 2 = 0.693\,147$ as the generating value: a difference of 0.4 % in the rate constant. Neither value is incorrect, since they answer different questions, but the gap matters when a rate constant is reported. `fitCurve` therefore exposes a `linearised` flag, true for the exponential and power models only, so callers may surface the distinction. The curve fitter no longer uses these fits: it fits the typed equation by Levenberg–Marquardt on untransformed data (`fitModel` in `nonlinear-fit`), matching `scipy.optimize.curve_fit` to better than $1 \times 10^{-7}$ in the fitted values and to about $1 \times 10^{-6}$ in the standard errors and the confidence band. The log-space fits remain in `curve-fitting` for the npm package. A dataset containing points a model cannot transform is refused with a message naming the requirement (exponential needs every $y > 0$, power every $x > 0$ and $y > 0$, logarithmic only $x > 0$) rather than having the offending points silently dropped.

### 2.2.4 Digitisation, unit conversion and authoring utilities

Recovering values from a published figure requires a two-point affine calibration per axis. On a logarithmic axis the interpolation must be done in logarithmic space,

$$v(p) = 10^{\log_{10} v_1 + \dfrac{(p - p_1)\,(\log_{10} v_2 - \log_{10} v_1)}{p_2 - p_1}}, \tag{3}$$

because a decade occupies a constant pixel distance. Treating a logarithmic axis linearly is the commonest error in figure digitisation, and it yields values that are plausible rather than obviously wrong. The module also reports the data-space uncertainty of a one-pixel positioning error, since a digitised point's precision is set by the figure's resolution, not by the decimal places the export prints.

Sixty-four units across ten categories are each defined by their ratio to a category base unit, so conversion reduces to $x_{\text{to}} = x_{\text{from}} \cdot f_{\text{to}} / f_{\text{from}}$. The table is therefore linear rather than quadratic in the number of units, and every entry is independently checkable against its source: factors are CODATA 2018 values [23], validated against `scipy.constants` from SciPy 1.17.1, which ships the 2022 adjustment; the two agree to a relative $1 \times 10^{-10}$, far below the tolerance of any conversion the tools perform. Temperature is handled separately, being affine rather than multiplicative.

Two aspects of scheduler scripts dominate wasted allocation. First, the resource model differs by engine: GROMACS is threaded and wants one MPI rank per node with many CPUs per task, whereas LAMMPS [2] is MPI-parallel and wants many ranks per node. The wrong shape either idles most of a node or oversubscribes it. Second, and less widely appreciated, the SLURM `--mem` directive in an array job specifies memory *per task*, not per job: a request of 32 GB across a hundred-task array asks the scheduler for 3.2 TB concurrently. The generator computes the true in-flight total and reports it. Generated scripts resume from the last checkpoint rather than silently restarting from $t = 0$, a failure mode that consumes an entire allocation while appearing to succeed. Warnings are returned as structured objects, leaving presentation to the caller.

A single resource request is emitted for any of four schedulers, SLURM, PBS Professional/OpenPBS, LSF and Grid Engine, because the same allocation is expressed by different directives, environment variables and launch commands on each; values that are site-dependent, among them the queue name, the name of the parallel environment and the resource under which GPUs are requested, are marked as such in the generated script rather than guessed.

The remaining modules support manuscript preparation: LaTeX and Markdown table generation with correct escaping, ISO-4 journal title abbreviation, and BibTeX deduplication by union-find over normalised DOIs and titles, in which entries with different DOIs are never merged.

### 2.2.5 Defects exposed by extraction

Separating computation from presentation subjected the numerical code to independent verification for the first time, and four defects were identified that had propagated into reported output. First, skewness and kurtosis are defined against the *population* standard deviation,

$$G_1 = \frac{\sqrt{n(n-1)}}{n-2} \cdot \frac{1}{n} \sum_{i=1}^{n} \left( \frac{x_i - \bar{x}}{\sigma_{\text{pop}}} \right)^3, \qquad \sigma_{\text{pop}} = \sqrt{\frac{1}{n} \sum_{i} (x_i - \bar{x})^2}, \tag{4}$$

whereas the original implementation used the Bessel-corrected sample standard deviation, deflating skewness by a factor of $((n-1)/n)^{3/2}$, approximately 15% at $n = 10$. Because both moments feed the D'Agostino–Pearson statistic, every normality $p$-value the tool reported was affected. Second, all upper-tail probabilities were computed by subtraction and floored at zero for strong effects. Third, haem iron was assigned the mass of fluorine (18.998 Da rather than 55.845 Da), selenomethionine selenium the mass of sulfur, and numeric-prefixed hydrogens no mass at all, so molecular weights and centres of mass computed for metalloproteins were correspondingly wrong. Fourth, the D'Agostino–Pearson transformations are defined on the biased moment ratios $\sqrt{b_1} = m_3 / m_2^{3/2}$ and $b_2 = m_4 / m_2^2$, but the sample-size-adjusted $G_1$ of Eq. 4 was passed into the skewness transform instead. The adjustment inflates $\sqrt{b_1}$ by exactly $\sqrt{n(n-1)}/(n-2)$: a factor of 1.247 at $n = 8$, 1.186 at $n = 10$ and 1.039 at $n = 40$. The error therefore vanishes on symmetric data and grows with skew. Because that factor exceeds unity for every $n$, the inflated statistic is always the larger and the reported $p$ always the smaller, so the bias

is systematically anti-conservative rather than merely noisy: the verdict at $\alpha = 0.05$ can only ever move from retaining normality to rejecting it, which is precisely the branch that redirects a user from Welch's test to Mann–Whitney. This defect is the instructive one, because an independent reference was already in place and already disagreed. The residual had been absorbed by loosening the assertion to three decimal places and blaming SciPy's kurtosis transform; that transform is identical, and the fixture was nearly symmetric ($g_1 = -0.0063$), so the inflation multiplied approximately zero. The reference value was correct and the implementation was not. $K^2$ now reproduces `scipy.stats.normaltest` to twelve significant figures across $8 \le n \le 1000$, the full range over which the $S_U$ approximation is tabulated [17] and the provenance of the module's $n \ge 8$ guard, while `skewness()` continues to report $G_1$, the appropriate statistic to publish and the one matching `scipy.stats.skew(bias=False)`.

Each defect is corrected and covered by a regression test. Users who generated figures with earlier versions of these tools should re-check any reported skewness, normality $p$-value, or metalloprotein mass.

### 2.3 Sample code snippets analysis

Listing 1 contains the whole of the environment-specific code in the library. The package's Node.js entry point registers the vendored bundles, and a browser page registers the globals its script tags installed; after that the identical modules run unchanged in both, and a script needs no registration of its own.

```
// Node.js: src/core/node.js, the package entry
// under Node, loads the bundles.
import { createRequire } from 'node:module';
import { registerVendor } from './vendor.js';

const require = createRequire(import.meta.url);
const dep = (f) => require('../../js/dependencies/' + f);
registerVendor({
  jStat: dep('jstat.min.js'),
  Papa: dep('papaparse.min.js'),
  regression: dep('regression.min.js'),
  bibtexParse: dep('bibtexParse.min.js')
});
export * from './index.js';

// Browser: the <script> tags set the globals.
import { registerFromGlobals } from './src/core/index.js';
registerFromGlobals();
```

**Listing 1.** Registration of the vendored libraries by the Node.js entry point and in the browser.

## 3 Illustrative examples

The example below is the post-simulation triage that motivated the toolkit: confirm that a production run has equilibrated, characterise the final structure, test a replicate comparison and queue a continuation. Every step is available interactively as a tool page; the scripted form is shown because it is the reproducible one.

Listing 2 reads a GROMACS RMSD series. The parser returns the numeric matrix together with the title and per-series legends recovered from the `@` directives, so each column is reported under its own observable name rather than by index.

```
import { parseXvg, extractColumn, columnStats } from 'stemkit-core';
import { readFileSync } from 'fs';

const { matrix, headers, title } = parseXvg(readFileSync('rmsd.xvg', 'utf8'));
// title   -> "RMSD & Radius of Gyration"
// headers -> ["Time (ps)", "Backbone RMSD", "Rg"]

headers.slice(1).forEach((name, i) => {
  const s = columnStats(extractColumn(matrix, i + 1));
  console.log(`${name}: ${s.mean.toFixed(4)} +/- ${s.std.toFixed(4)} nm`);
});
```

**Listing 2.** Parsing a GROMACS RMSD trajectory and summarising each series.

Listing 3 takes the final coordinates and computes the mass-weighted centre of mass and radius of gyration. Because the input is a `.gro` file its coordinates are in nanometres, and the conversion to Å is requested explicitly rather than assumed.

```
import { parseStructure, centreOfMass, radiusOfGyration, convert }
  from 'stemkit-core';

const frame = parseStructure(readFileSync('final.gro', 'utf8'), 'gro');
const com = centreOfMass(frame.atoms);   // {x, y, z} in nm, and the total mass
const rg  = radiusOfGyration(frame.atoms); // nm, mass-weighted about the COM

console.log(`Rg = ${convert(rg, 'length', 'nm', 'angstrom').toFixed(2)} A`);
```

**Listing 3.** Geometry of the final frame, with an explicit unit conversion.

Listing 4 compares two sets of replicate measurements. `recommendTest` runs the assumption checks before any test and names the one they point to, with its reason, so a violated assumption is visible at the point of use rather than discovered later; here a failed normality check in one group points to Mann–Whitney. The recommendation is advice, and the caller runs the test.

```
import { recommendTest, independentTTest, mannWhitneyU } from 'stemkit-core';

const advice = recommendTest({ design: 'independent',
                               groups: [wildType, mutant],
                               names: ['wild type', 'mutant'] });
console.log(advice.reason);   // why this test, e.g. which group is not normal

const r = advice.test === 'mann-whitney'
  ? mannWhitneyU(wildType, mutant)        // r.U, r.p, r.rankBiserial
  : independentTTest(wildType, mutant);   // Welch: r.t, r.df, r.p, r.g, r.ci
```

**Listing 4.** A replicate comparison guided by the assumption checks.

Finally, Listing 5 generates the submission script for the continuation run. The engine argument selects the GROMACS resource shape (one rank per node, many CPUs per task) and the returned warnings report the true in-flight memory total for the array as well as any request the scheduler is likely to reject.

```
import { generateScript } from 'stemkit-core';

const { script, warnings } = generateScript({
  engine: 'gromacs', jobName: 'prod_md', partition: 'gpu',
  nodes: 1, gpus: 1, cpusPerTask: 16,
  walltime: '24:00:00', memory: '32G',
```

```
  modules: ['gcc/11.3', 'cuda/12.1', 'gromacs/2023.3'],
  tpr: 'md.tpr', deffnm: 'md', maxh: 23.5
});

warnings.forEach(w => console.warn(`[${w.level}] ${w.message}`));
```

**Listing 5.** Generating a GROMACS submission script with resource warnings.

The same four steps through the browser produce identical numbers, since they call the same functions. The plotting steps also emit a matplotlib script that regenerates the figure from the same data. Figures 2 and 3 show five of the tools in use.

## 4 Impact

The architecture's most direct effect is on work that server-based tools cannot do at all. Where upload is prohibited, the relevant comparison is not with a faster tool but with doing the calculation by hand.

For research questions already being pursued, the gain is in reproducibility and in the reliability of the numbers. An analysis first explored by clicking can be captured as a script and version-pinned without being rewritten in another language, which is the step at which reproducibility is most often lost, and the emitted matplotlib scripts close the same loop for figures. Every reported statistic is checked against SciPy, NumPy or statsmodels rather than against the author's own expectations, and the suite is meant to be run by users evaluating the software: a library whose numerical claims cannot be checked offers reproducibility only in principle.

The defects found during extraction (Section 2.2.5) carry a wider lesson, and are the most useful outcome of this work. All four had survived in software that ran without error and produced plausible numbers. Three would not have been caught by testing the code against expectations derived from the same source; each surfaced only when results were compared with an independent implementation. The fourth is the more uncomfortable case, because the independent comparison had already been made and had already disagreed: the tolerance was widened until the test passed and the residual was explained away. An independent reference is necessary but not sufficient. A disagreement with one must be treated as a defect until shown otherwise, and a loosened tolerance is where that demonstration usually fails to happen. A 15% error in a reported skewness is not visible by inspection, and neither is a $p$-value floored at zero, which looks like a very strong result. Authors of small, rarely reviewed in-house analysis code would be well advised to check their numerics against an independent implementation before reporting output.

In daily practice, the change is the removal of a setup step from tasks that previously carried one. Checking whether an RMSD has converged, or whether a replicate difference survives an assumption check, becomes an operation performed at the moment the question arises rather than one deferred until an environment is available. The tools are also usable in teaching and in shared computing environments where installation is not an option, since a URL is the only prerequisite. Each of the research tools has a narrated, captioned video tutorial that works through the tool step by step on an example (the MD workflow generator has one for each engine); each is shown on its tool's page and the set is collected in a YouTube playlist (`https://www.youtube.com/playlist?list=PLAqgfbqFf32M`).

STEMKit is MIT-licensed, hosted at `https://stemkit.net` and archived on Zenodo [29]. The software was first archived in July 2026 and published on npm with version 0.2.0, so adoption data (downloads, unique users, citing publications) does not yet exist. The arguments

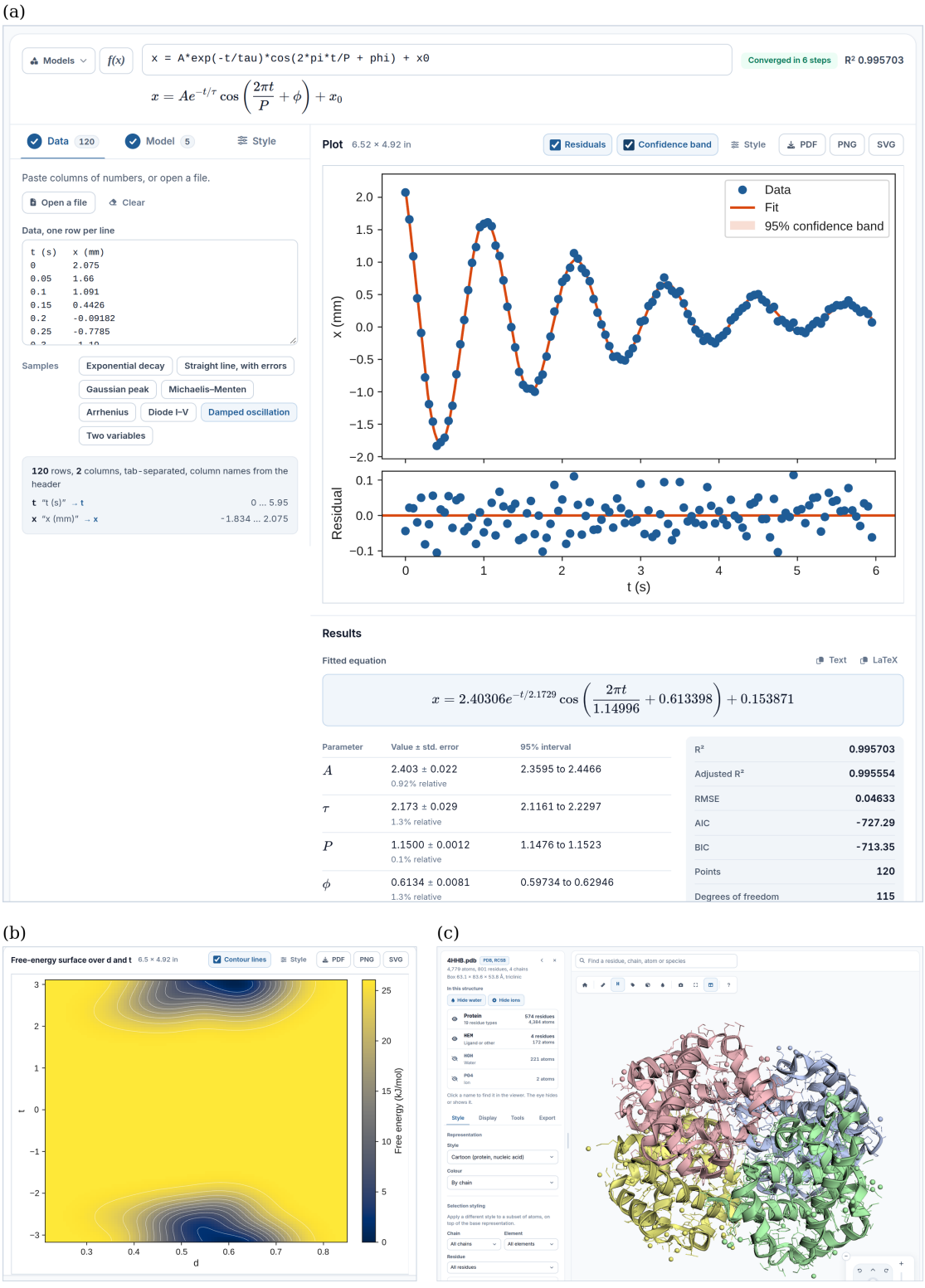
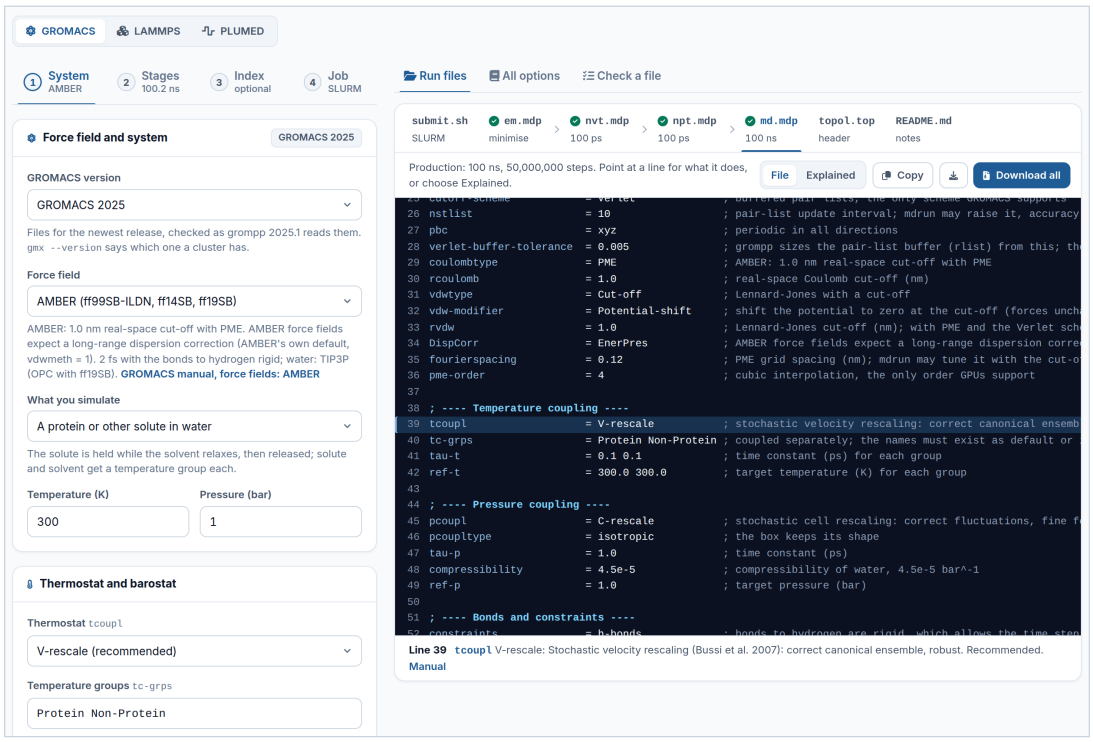



**Figure 2.** Three of the tools, each with its own sample or a public structure. (a) The curve fitter: a damped oscillation typed as an equation and fitted by Levenberg–Marquardt, with its 95% confidence band, the residuals below, and each parameter with its standard error and interval. (b) The PLUMED run analysis: the free-energy surface of a two-variable well-tempered metadynamics run, summed from its HILLS file as `plumed sum_hills` sums it. (c) The structure inspector: haemoglobin (PDB 4HHB) fetched from the RCSB, drawn as a cartoon coloured by chain, with its four haem groups.

(a)




(b)

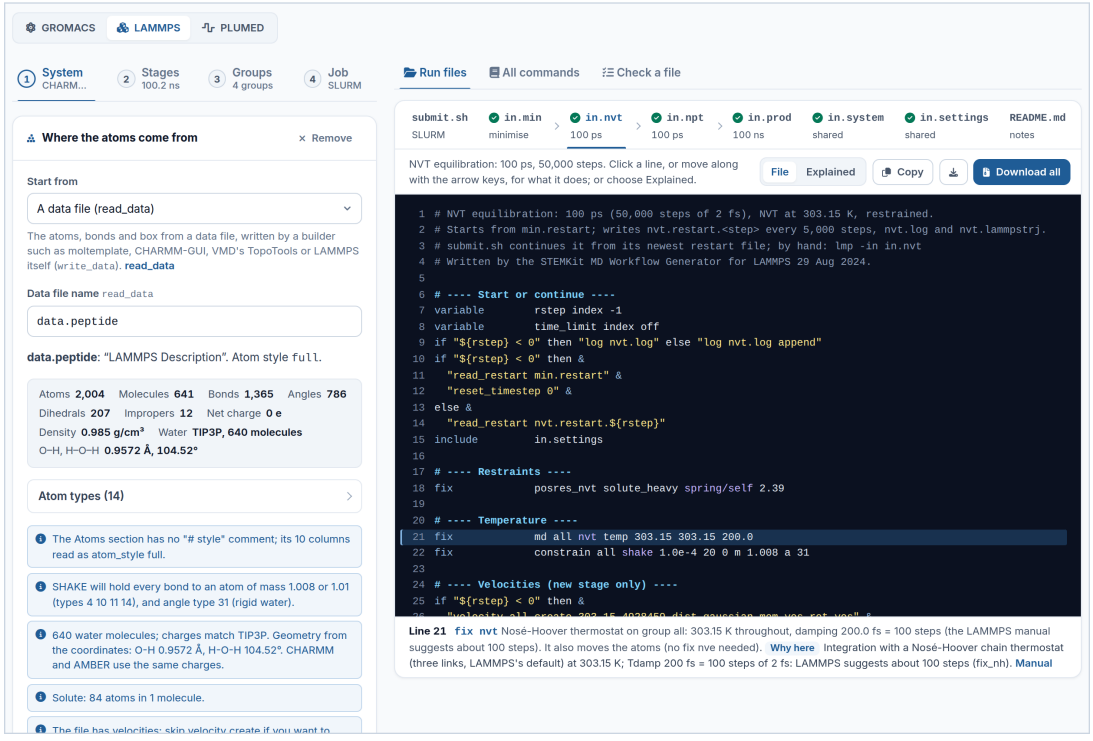



**Figure 3.** Setting up simulations. (a) GROMACS: the System step and the production `md.mdp`, with the selected line (`tcoupl`) explained beneath the file; a tick on each file's tab means it passes the checker. (b) LAMMPS: a data file read in the browser (the LAMMPS peptide example, 2004 atoms, its TIP3P water found from the topology) and the NVT input written for it, with the selected `fix nvt` line explained.

above are accordingly claims about what the design makes possible, not evidence of uptake, and should be read as such. The software is not used in a commercial setting and has not led to a spin-off company; the client-side design does, however, make it directly applicable to industrial research groups whose data-handling policies preclude uploading results to a third-party service.

## 5 Conclusions

STEMKit is a suite of 18 browser-based tools for computational chemistry built on a tested, dependency-injected JavaScript core of 42 domain modules. The architecture addresses three constraints simultaneously that existing tooling addresses only in pairs: it requires no installation, transmits no data and remains scriptable.

Both properties are structural rather than contractual: computation occurs entirely within the browser, and the modules behind the interface are the ones that run under Node.js. The 4989-test suite validates every numerical result against an independent reference, and the four defects that this strategy exposed (deflated standardised moments, tail probabilities floored at zero, misassigned elements in metalloproteins, and an adjusted moment fed into a transform defined on the unadjusted one) are documented, corrected and covered by regression tests. Future work will extend format coverage in the trajectory and structure modules.

## Declaration of generative AI use

The author conceived STEMKit and set its scope and design: which tools to build, what each must do, the methods they use, and the independent references and programs each must agree with. The author also tested the tools and decided what was released. Claude (Anthropic), used through Claude Code, assisted with the implementation, writing code, tests and documentation to the author's specifications, and helped draft and revise this manuscript. Correctness does not rest on the model: every numerical routine is tested against an independent reference or against the programs themselves (Section 2.1), those tests run in continuous integration, and the author reviewed the results. The author takes full responsibility for the software and for the content of this manuscript; the tool is not an author.

## Acknowledgements

I thank the maintainers of jStat, Papa Parse, regression.js, bibtex-parse-js, Plotly, KaTeX, MathLive and 3Dmol.js, whose libraries STEMKit builds upon, and the GROMACS, PLUMED and LAMMPS developers, from whose source code and documentation the bundled reference tables are generated [1, 4, 30, 2].

## Declaration of competing interest

The author declares no competing financial or non-financial interests.

## References

[1] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers, *SoftwareX* **1–2**, 19–25 (2015). https://doi.org/10.1016/j.softx.2015.06.001

[2] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton, LAMMPS — a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, *Comput. Phys. Commun.* **271**, 108171 (2022). https://doi.org/10.1016/j.cpc.2021.108171

[3] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, and O. Beckstein, MDAnalysis: A toolkit for the analysis of molecular dynamics simulations, *J. Comput. Chem.* **32**(10), 2319–2327 (2011). https://doi.org/10.1002/jcc.21787

[4] G. A. Tribello, M. Bonomi, D. Branduardi, C. Camilloni, and G. Bussi, PLUMED 2: New feathers for an old bird, *Comput. Phys. Commun.* **185**(2), 604–613 (2014). https://doi.org/10.1016/j.cpc.2013.09.018

[5] N. Rego and D. Koes, 3Dmol.js: molecular visualization with WebGL, *Bioinformatics* **31**(8), 1322–1324 (2015). https://doi.org/10.1093/bioinformatics/btu829

[6] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, *et al.*, SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nat. Methods* **17**(3), 261–272 (2020). https://doi.org/10.1038/s41592-019-0686-2

[7] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, *et al.*, Array programming with NumPy, *Nature* **585**(7825), 357–362 (2020). https://doi.org/10.1038/s41586-020-2649-2

[8] S. Seabold and J. Perktold, Statsmodels: econometric and statistical modeling with Python, in: *Proceedings of the 9th Python in Science Conference*, 2010, pp. 92–96. https://doi.org/10.25080/Majora-92bf1922-011

[9] B. L. Welch, The generalization of 'Student's' problem when several different population variances are involved, *Biometrika* **34**(1–2), 28–35 (1947). https://doi.org/10.1093/biomet/34.1-2.28

[10] B. L. Welch, On the comparison of several mean values: an alternative approach, *Biometrika* **38**(3–4), 330–336 (1951). https://doi.org/10.1093/biomet/38.3-4.330

[11] C. Spearman, The proof and measurement of association between two things, *Am. J. Psychol.* **15**(1), 72–101 (1904). https://doi.org/10.2307/1412159

[12] W. H. Kruskal and W. A. Wallis, Use of ranks in one-criterion variance analysis, *J. Am. Stat. Assoc.* **47**(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441

[13] J. W. Tukey, Comparing individual means in the analysis of variance, *Biometrics* **5**(2), 99–114 (1949). https://doi.org/10.2307/3001913

[14] C. Y. Kramer, Extension of multiple range tests to group means with unequal numbers of replications, *Biometrics* **12**(3), 307–310 (1956). https://doi.org/10.2307/3001469

[15] P. A. Games and J. F. Howell, Pairwise multiple comparison procedures with unequal n's and/or variances: a Monte Carlo study, *J. Educ. Stat.* **1**(2), 113–125 (1976). https://doi.org/10.3102/10769986001002113

[16] O. J. Dunn, Multiple comparisons using rank sums, *Technometrics* **6**(3), 241–252 (1964). https://doi.org/10.1080/00401706.1964.10490181

[17] R. B. D'Agostino and E. S. Pearson, Tests for departure from normality. Empirical results for the distributions of $b_2$ and $\sqrt{b_1}$, *Biometrika* **60**(3), 613–622 (1973). https://doi.org/10.1093/biomet/60.3.613

[18] M. B. Brown and A. B. Forsythe, Robust tests for the equality of variances, *J. Am. Stat. Assoc.* **69**(346), 364–367 (1974). https://doi.org/10.1080/01621459.1974.10482955

[19] S. Holm, A simple sequentially rejective multiple test procedure, *Scand. J. Stat.* **6**(2), 65–70 (1979).

[20] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*, ASQC Quality Press, Milwaukee, WI (1993).

[21] F. E. Grubbs, Procedures for detecting outlying observations in samples, *Technometrics* **11**(1), 1–21 (1969). https://doi.org/10.1080/00401706.1969.10490657

[22] D. G. Bonett and T. A. Wright, Sample size requirements for estimating Pearson, Kendall and Spearman correlations, *Psychometrika* **65**(1), 23–28 (2000). https://doi.org/10.1007/BF02294183

[23] E. Tiesinga, P. J. Mohr, D. B. Newell, and B. N. Taylor, CODATA recommended values of the fundamental physical constants: 2018, *Rev. Mod. Phys.* **93**(2), 025010 (2021). https://doi.org/10.1103/RevModPhys.93.025010

[24] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed., Cambridge University Press, Cambridge (2007).

[25] W. McKinney, Data structures for statistical computing in Python, in: *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

[26] J. D. Hunter, Matplotlib: A 2D graphics environment, *Comput. Sci. Eng.* **9**(3), 90–95 (2007). https://doi.org/10.1109/MCSE.2007.55

[27] R. B. D'Agostino, Transformation to normality of the null distribution of $g_1$, *Biometrika* **57**(3), 679–681 (1970). https://doi.org/10.1093/biomet/57.3.679

[28] F. J. Anscombe and W. J. Glynn, Distribution of the kurtosis statistic $b_2$ for normal samples, *Biometrika* **70**(1), 227–234 (1983). https://doi.org/10.1093/biomet/70.1.227

[29] O. M. Daramola, STEMKit: A client-side toolkit for computational chemistry and scientific data analysis (v0.5.1), Zenodo (2026). https://doi.org/10.5281/zenodo.21543112

[30] The PLUMED consortium, Promoting transparency and reproducibility in enhanced molecular simulations, *Nat. Methods* **16**(8), 670–673 (2019). https://doi.org/10.1038/s41592-019-0506-8